Avril 2026

Détection de fraude par carte bancaire

Projet Machine Learning — Polytech Semestre 4

Jegham Mohamed **Ben Mansour Louay**

Parcours Machine Learning

Plan

1. Contexte et données

2. Méthodologie

3. Résultats

4. Sorties du projet et démonstration

5. Limites et perspectives

6. Conclusion

Contexte et données

1

Contexte

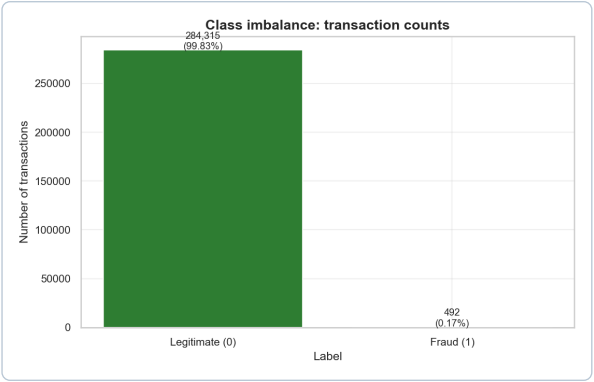
Enjeux de la fraude bancaire

Volume élevé de transactions $\rightarrow$ nécessité d'un **scoring automatique** robuste.

- Pertes financières, obligations réglementaires et confiance client en jeu.
- Classification binaire : **légitime** *vs* **frauduleux** sur données tabulaires réelles.
- Benchmark académique standard : dataset ULB / Kaggle.

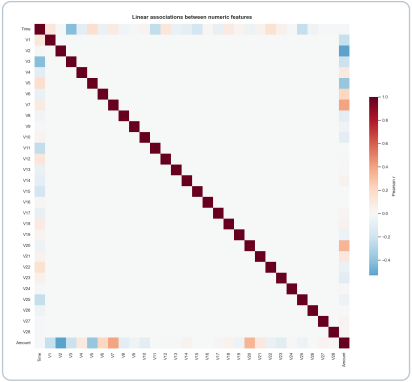
Jeu de données — ULB / Kaggle

- Fichier `creditcard.csv` : $\approx 285\,000$ transactions.
- Variables : `Time`, `Amount`, `V1`–`V28` (PCA anonymisée), cible `Class` $\in \{0, 1\}$.
- **Déséquilibre sévère** : $< 0.2\,\%$ de fraudes.
- L'exactitude brute est *trompeuse* $\rightarrow$ F1, ROC-AUC, Précision, Rappel.

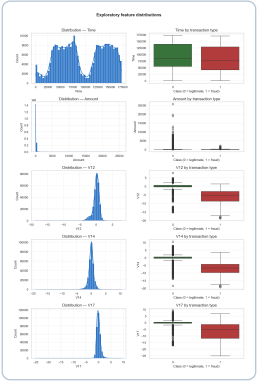


Exploration : corrélations et distributions

Corrélations (heatmap)



Distributions des features



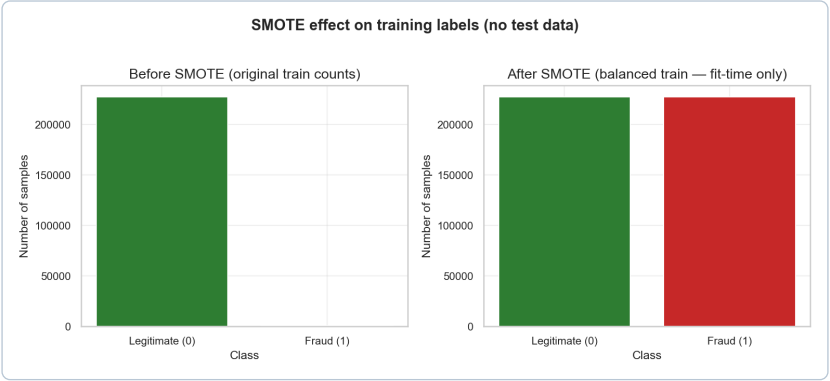
Méthodologie

2

Pipeline de traitement (vue d'ensemble)

1. **EDA** : distributions, corrélations, valeurs manquantes.
2. **Split temporel** : tri par `Time`, train = début, test = fin — plus réaliste.
3. **Features** : `log1p(Amount)`, ratios vs moyenne mobile causale (sans fuite).
4. **Sélection** : régularisation L1 (logistique) sur le train.
5. **Rééquilibrage** : standardisation + **SMOTE** (train uniquement, `imblearn`).
6. **Modèles** : LR, Arbre, RF (GridSearch), XGBoost, LightGBM.
7. **Seuil optimal** : courbe PR + coût métier $C_{\mathrm{FN}} \cdot \mathrm{FN} + C_{\mathrm{FP}} \cdot \mathrm{FP}$.
8. **Post-traitement** : calibration, dérive (KS, PSI), SHAP.
9. **Sauvegarde** : `best_model.pkl`, liste des variables `feature_columns.json` (25 colonnes après sélection).
10. **Démonstration** : application **Streamlit** (seuil, graphiques Plotly, import CSV de test).

SMOTE : rééquilibrage sur le train



Résultats

3

Comparaison des modèles (jeu de test)

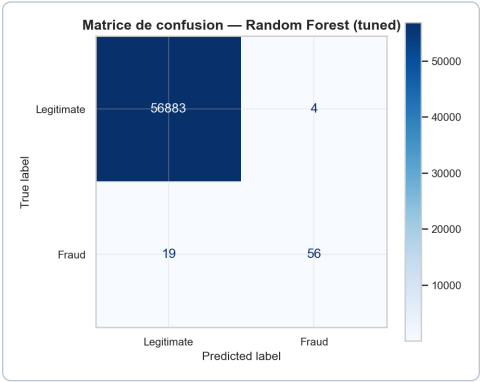
Métriques issues de `model_results.csv` (dernière exécution du notebook).

Modèle	Acc.	Préc.	Rappel	F1	AUC
Régression logistique	0,984	0,065	0,853	0,121	0,981
Arbre de décision	0,993	0,132	0,773	0,225	0,820
Random Forest (tuné)	**1,000**	**0,933**	**0,747**	**0,830**	**0,964**
XGBoost	0,999	0,652	0,773	0,707	0,980
LightGBM	0,999	0,637	0,773	0,699	0,982

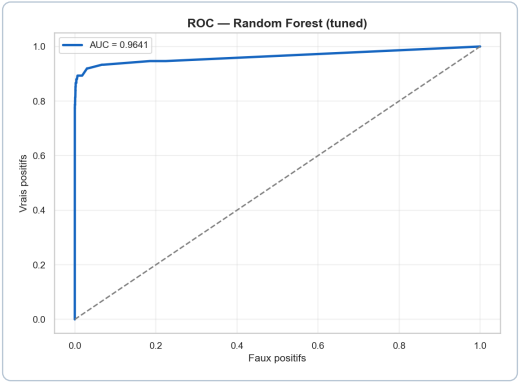
Lecture

Meilleur **F1** sur ce run : **Random Forest** (ligne verte). Les valeurs se mettent à jour automatiquement après `02_Modeling.ipynb`.

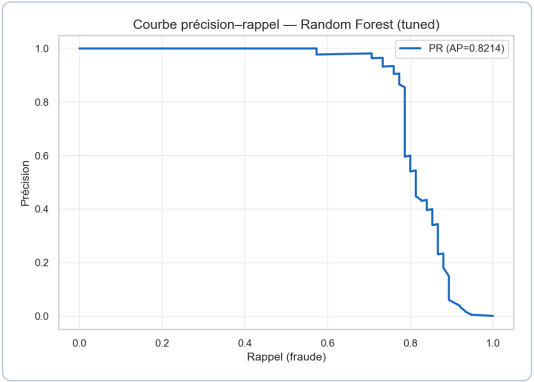
Matrice de confusion : modèle retenu



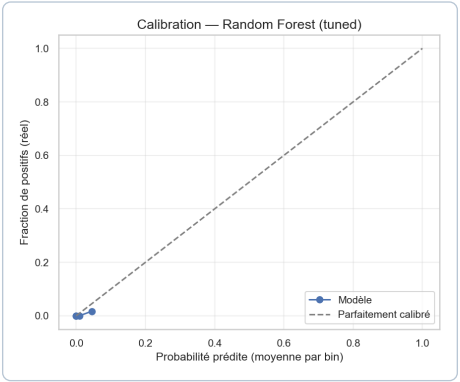
Courbe ROC


ROC — Random Forest (tuned)
AUC = 0.9641
Vrais positifs
Faux positifs
0.0
0.2
0.4
0.6
0.8
1.0

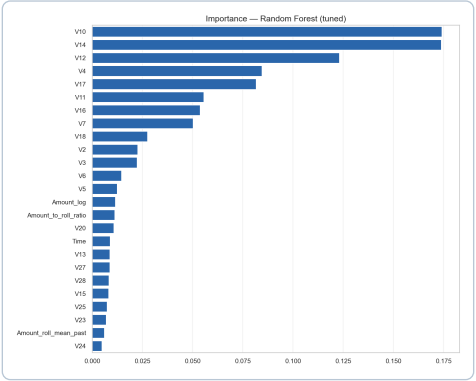
Précision–Rappel et choix de seuil



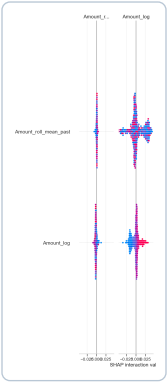
Calibration des probabilités



Importance des variables (modèle retenu)



Explicabilité : SHAP (summary plot)



Sorties du projet et démonstration

4

Artefacts dans

Fichiers générés par le pipeline

- **Modèle** : `best_model.pkl`
- **Entrées du scoreur** : `feature_columns.json` (25 variables retenues)
- **Tableau de synthèse** : `model_results.csv`
- **Dérive** : `drift_report.csv`
- **Figures** : ROC, PR, calibration, SHAP, matrices...

Cohérence

L'interface Streamlit charge le même pipeline et les mêmes noms de colonnes que le notebook — pas de décalage de schéma.

Relancer `02_Modeling.ipynb` après toute modification du flux pour régénérer ces fichiers.

Dérive train *vs* test (`drift_report.csv`)

Extrait des indicateurs exportés (KS, PSI) — *valeurs typiques d'une exécution récente* :

Variable	KS	p-val. KS	PSI
`Amount_log`	0,053	≈ 0	0,014
`Time`	1,000	≈ 0	11,51
`V14`	0,086	≈ 0	0,061
`V12`	0,088	≈ 0	0,151

Lecture

Split temporel : `Time` peut fortement diverger entre train et test (PSI élevé). Sur les `V*` et `Amount_log`, le PSI reste souvent modéré — à suivre en production (seuil courant PSI $> 0{,}2$).

Interface Streamlit — essai du modèle

- **Seuil** réglable + coût métier C_{FN}:C_{FP} aligné sur `src/config.py`.
- **Échantillon** : fenêtre aléatoire, depuis le début, ou décalage fixe — reproductibilité affichée, export CSV des prédictions.
- **Visualisation** : histogrammes Plotly des probabilités par vraie classe.
- **Import CSV** : fichier avec `Time`, `Amount`, `V1`–`V28` ; historique causal du jeu pour les moyennes mobiles ; résultats téléchargeables.

Lancement

`streamlit run streamlit_app.py` *(depuis la racine du projet)*

Limites et perspectives

5

Limites et perspectives

Limites

- Données de 2013 : risque de dérive en production.
- Variables PCA anonymisées : interprétation métier restreinte.

Pistes

- CatBoost, TabNet.
- Coûts métier personnalisés.
- MLOps : monitoring PSI/KS continu.

Conclusion

Conclusion

Livrables techniques

`requirements.txt` · `01_EDA` / `02_Modeling` · `src/` · `*` · `streamlit_app.py` · tests `pytest`

- Automatisation : `Makefile`, `tasks.ps1` ; CI GitHub Actions.
- **Random Forest (GridSearch)** : meilleur F1 sur le jeu de test actuel (**F1** $\approx 0{,}83$, AUC-ROC $\approx 0{,}96$) — voir `model_results.csv`.
- **Démonstration interactive** : scoring, seuil, import CSV et figures Plotly via Streamlit.
- Le modèle reste un **outil d'aide à la décision** : seuil et arbitrage FN/FP sont métier.

Merci de votre attention

Dataset Kaggle — ULB Credit Card Fraud

Jegham Mohamed Ben Mansour Louay · Parcours Machine Learning